Edit the venue, add a description, upload an image, and save the changes. If you check out your uploads directory, you'll find the image you uploaded. Wait a second—what if you upload another file with the same name? What happens? Django takes care of that for you: when it saves a file to the uploads folder, it checks for naming conflicts, renaming the file with a random ending when necessary. It even makes sure the resulting filename is shorter than the maximum allowed by the field.

NOTE By default, `FileField` and `ImageField` use a 100-character varchar to store your filename. You can change this with the `max_length` argument, similarly to `CharField`.

Dynamically changing the upload directory

Both the `FileField` and `ImageField` have an optional argument called `upload_to`, which you can use to alter the name of the directory a file gets uploaded into. This argument supports the string notation of the `strftime()` function, allowing you to set the upload directory based on the current date, as shown in the following example:

```
class Person(models.Model):
    picture = models.ImageField(upload_to="%Y/%m/%d/")
```

The upload is always relative to the `MEDIA_ROOT` value. Using a date stamp can make your upload directory cleaner, with fewer files in the root.

Instead of using a string, `upload_to` also supports calling a function. You pass a reference to the function, which the field then invokes when it needs to create a path. Your code is responsible for providing the path for the uploaded file. Remember to use a reference. You don't want to call the function yourself; you want the field to do it for you. The path creating function takes two arguments: an instance of the model object and the name of the file being saved.

```
def user_path(instance, filename):
    return f"user{instance.owner.id}/{filename}"
```

A custom function to determine the upload path

```
class Person(models.Model):
    owner = models.ForeignKey(user, on_delete=models.CASCADE)
    picture = models.ImageField(upload_to=user_path)
```

Set the upload_to argument to a reference to the custom path function.

There is a chicken-and-egg problem with this code, though: with a newly created `Person` object, the `picture` field gets populated before the `Person` gets saved. This means the `instance` argument to the function will not have an ID when the object gets created. You also don't have access to the user's information unless that is also part of the model. Basing the filename on the logged user isn't possible through this mechanism, unless that information is stored in the model.